

Three Server Build, Nexus, and Deployment Architecture Documentation

1. Project Overview

This project demonstrates a three-server architecture used for building, storing, and deploying an application manually using different servers.

The architecture is designed to separate build, artifact storage, and deployment activities across different servers for better management and organization.

The workflow includes:

1. Cloning source code from Git repository
 2. Building the application using Node.js and NPM
 3. Creating build artifacts
 4. Storing the build artifact in Nexus Repository
 5. Downloading and deploying the build artifact to the deployment server
-

2. Architecture Overview

Three Server Architecture

Server 1 – Build Server

The Build Server is responsible for:

- Cloning the source code from Git
- Installing dependencies
- Building the application
- Creating deployment artifacts
- Uploading the artifact to Nexus Repository

Server 2 – Nexus Repository Server

The Nexus Server acts as an artifact repository.

Responsibilities:

- Storing build artifacts
- Managing version control of artifacts
- Providing centralized artifact access
- Allowing deployment server to download build packages

Server 3 – Deployment Server

The Deployment Server is responsible for:

- Downloading build artifacts from Nexus
 - Extracting the application package
 - Deploying the application
 - Running the application in the target environment
-

3. Tools and Technologies Used

Tool / Technology	Purpose
Git	Source Code Management
Node.js	JavaScript Runtime Environment
NPM	Package Management
Angular / React / Node Application	Application Build
Nexus Repository	Artifact Storage
Linux Server	Hosting Environment
Curl	Uploading artifacts to Nexus
Wget	Downloading artifacts from Nexus
Zip	Packaging build files

4. Manual Build and Deployment Workflow

Step-by-Step Process

Step 1: Clone Source Code from Git

The source code is cloned from the Git repository into the Build Server.

Command

```
git clone https://github.com/Ai-TechNov/AngularCalculator.git
```

```
ubuntu@Build-server:~$ git clone https://github.com/Akiranred/AngularCalculator.git
Cloning into 'AngularCalculator'...
remote: Enumerating objects: 39, done.
remote: Total 39 (delta 0), reused 0 (delta 0), pack-reused 39 (from 1)
Receiving objects: 100% (39/39), 107.66 KiB | 8.28 MiB/s, done.
ubuntu@Build-server:~$ |
```

Step 2: Install Node.js and NPM

Node.js and NPM are installed on the Build Server.

Install Node.js

```
sudo apt update  
sudo apt install nodejs -y
```

Install NPM

```
sudo apt install npm -y
```

Verify Installation

```
node -v  
npm -v
```

```
ubuntu@Build-server:~/AngularCalculator$ npm --version  
9.2.0  
ubuntu@Build-server:~/AngularCalculator$ nodejs --version  
v22.22.1  
ubuntu@Build-server:~/AngularCalculator$ |
```

Step 3: Install Project Dependencies

After cloning the repository, install all project dependencies.

Command

```
npm install
```

This command installs all packages mentioned in package.json.

Sudo npm install -g @angular/cli

```
ubuntu@Build-server:~$ sudo npm install -g @angular/cli  
  
added 288 packages in 10s  
  
70 packages are looking for funding  
  run 'npm fund' for details  
ubuntu@Build-server:~$ npm install
```

Step 4: Build the Application

The application is built using the build command.

Angular Build

ng build

```
ubuntu@Build-server:~/AngularCalculator$ export NODE_OPTIONS=--openssl-legacy-provider
ubuntu@Build-server:~/AngularCalculator$ ng build
10% building 4/4 modules 0 active(node:9717) [DEP0111] DeprecationWarning: Access to process.binding('http_parser') is deprecated.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:9717) [DEP0111] DeprecationWarning: Access to process.binding('http_parser') is deprecated.
(node:9717) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.

Date: 2026-05-18T13:27:18.546Z
Hash: df285638c9c085024033
Time: 11222ms
chunk {es2015-polyfills} es2015-polyfills.js, es2015-polyfills.js.map (es2015-polyfills) 285 kB [initial] [rendered]
chunk {main} main.js, main.js.map (main) 14.7 kB [initial] [rendered]
chunk {polyfills} polyfills.js, polyfills.js.map (polyfills) 236 kB [initial] [rendered]
chunk {runtime} runtime.js, runtime.js.map (runtime) 6.08 kB [entry] [rendered]
chunk {styles} styles.js, styles.js.map (styles) 973 kB [initial] [rendered]
chunk {vendor} vendor.js, vendor.js.map (vendor) 3.2 MB [initial] [rendered]
ubuntu@Build-server:~/AngularCalculator$
```

The generated build files are stored inside the dist/ or build/ folder.

Step 5: Create ZIP File of Build

The build output is compressed into a ZIP file for easy transfer and deployment.

Command

```
zip -r calculator-v1.zip dist/
```

```
ubuntu@Build-server:~/AngularCalculator/dist$ zip -r calculator-v1.zip .
adding: angularCalc/ (stored 0%)
adding: angularCalc/polyfills.js (deflated 81%)
adding: angularCalc/index.html (deflated 58%)
adding: angularCalc/runtime.js.map (deflated 70%)
adding: angularCalc/styles.js (deflated 81%)
adding: angularCalc/styles.js.map (deflated 81%)
adding: angularCalc/main.js.map (deflated 69%)
adding: angularCalc/main.js (deflated 74%)
adding: angularCalc/polyfills.js.map (deflated 80%)
```

Step 6: Upload Build Artifact to Nexus Repository

The ZIP file is uploaded to Nexus Repository using curl command.

Command

```
curl -v -u admin:admin123 --upload-file calculator-v1.zip http://3.80.59.36:8081/repository/angular-builds/calculator-v1.zip
```

```

ubuntu@Build-server:~/AngularCalculator/dist$ curl -v -u admin:admin123 --upload-file calculator-v1.zip http://3.80.5
.36:8081/repository/angular-builds/calculator-v1.zip
* Trying 3.80.59.36:8081...
* Established connection to 3.80.59.36 (3.80.59.36 port 8081) from 172.31.26.161 port 39114
* using HTTP/1.x
* Server auth using Basic with user 'admin'
> PUT /repository/angular-builds/calculator-v1.zip HTTP/1.1
> Host: 3.80.59.36:8081
> Authorization: Basic YWRtaWw46YWRtaWw4xMjM=
> User-Agent: curl/8.18.0
> Accept: */*
> Content-Length: 1808694
> Expect: 100-continue
>
< HTTP/1.1 100 Continue
<
* upload completely sent off: 1808694 bytes
< HTTP/1.1 201 Created
< Server: Nexus/3.85.0-03 (COMMUNITY)
< X-Content-Type-Options: nosniff
< Content-Security-Policy: default-src http: data: blob: 'unsafe-inline'; script-src http: 'unsafe-inline' 'unsafe-eval'
< X-XSS-Protection: 1; mode=block
< Content-Length: 0
<
* Connection #0 to host 3.80.59.36:8081 left intact
ubuntu@Build-server:~/AngularCalculator/dist$

```

5. Nexus Repository Server Configuration

Install Nexus Repository

Update Packages

```
sudo apt update
```

Install Java

```
sudo apt install openjdk-17-jdk -y
```

Download Nexus

```
wget https://download.sonatype.com/nexus/3/latest-unix.zip
```

Unzip Nexus

```
Sudo unzip nexus
```

Start Nexus

```
cd nexus/bin
./nexus start
```

```

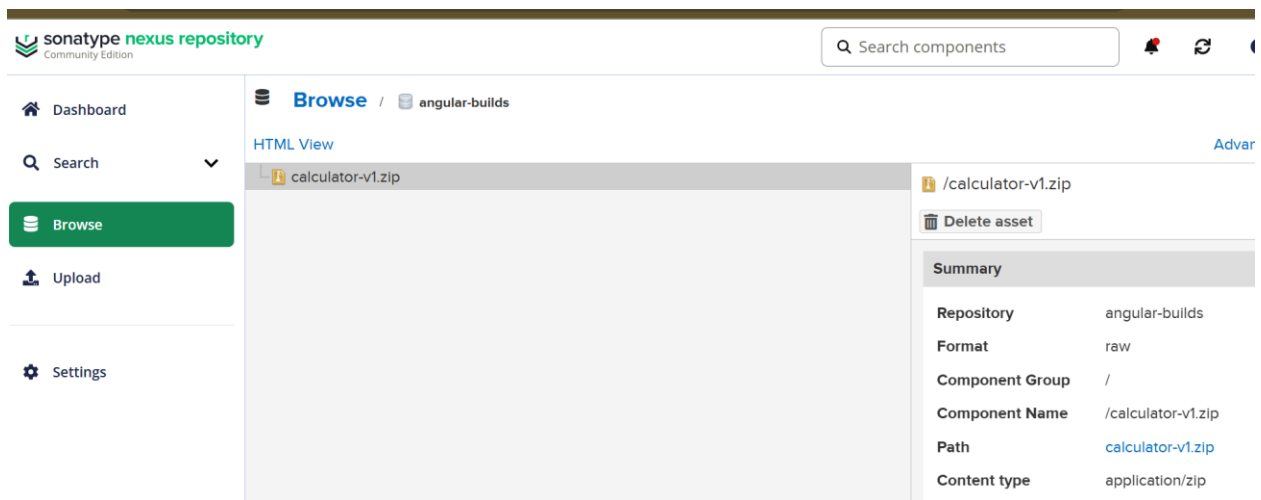
usage: ./nexus {start|stop|run|run -foreground|status|restart|force-reload}
ubuntu@Nexus-server:~/nexus-3.85.0-03/bin$ ./nexus start
Starting nexus
ubuntu@Nexus-server:~/nexus-3.85.0-03/bin$

```

Create Repository in Nexus

Inside Nexus UI:

1. Login to Nexus
2. Go to Settings
3. Select Repositories
4. Click Create Repository
5. Choose hosted repository
6. Enter repository name
7. Save configuration



6. Deployment Server Configuration

The Deployment Server downloads the build artifact from Nexus Repository.

Install Required Packages

Install Node.js and NPM

```
sudo apt update  
sudo apt install nodejs npm -y
```

```

ubuntu@Nexus-server:~$ sudo apt install nodejs npm
Installing:
  nodejs  npm

Installing dependencies:
  alacrity                node-graceful-fs
  build-essential         node-growl
  bzip2                   node-gyp
  cpp                     node-has-flag
  cpp-15                  node-has-unicode
  cpp-15-x86-64-linux-gnu node-has-value
  cpp-x86-64-linux-gnu    node-has-values
  dpkg-dev                node-hosted-git-info
  eslint                  node-http-proxy-agent
  fakeroot                 node-https-proxy-agent
  g++                     node-iconv-lite
  g++-15                  node-icss-utils
  g++-15-x86-64-linux-gnu node-ieee754
  g++-x86-64-linux-gnu    node-iferr
  gcc                     node-ignore
  gcc-15                  node-imurmurhash
  gcc-15-base              node-indent-string
  gcc-15-x86-64-linux-gnu node-inherits
  gcc-x86-64-linux-gnu     node-ini
  gyp                     node-interpret
  handlebars               node-ip

```

Verify Installation

```
node -v
```

```
npm -v
```

Download Artifact from Nexus Repository

The deployment server downloads the ZIP file using wget.

Command

```
wget http://3.80.59.36:8081/repository//application-build.zip
```

Extract ZIP File

Command

`unzip application-build.zip`

```
ubuntu@Deploy-server:~$ unzip calculator-v1.zip
Archive:  calculator-v1.zip
  creating: angularCalc/
  inflating: angularCalc/polyfills.js
  inflating: angularCalc/index.html
  inflating: angularCalc/runtime.js.map
  inflating: angularCalc/styles.js
  inflating: angularCalc/styles.js.map
  inflating: angularCalc/main.js.map
  inflating: angularCalc/main.js
  inflating: angularCalc/polyfills.js.map
  inflating: angularCalc/vendor.js
  inflating: angularCalc/es2015-polyfills.js
  inflating: angularCalc/runtime.js
  inflating: angularCalc/es2015-polyfills.js.map
  inflating: angularCalc/vendor.js.map
  inflating: angularCalc/favicon.ico
ubuntu@Deploy-server:~$ ls
```

Deploy the Application

Move the extracted files to the deployment directory.

`cp -r dist/* /var/www/html/`

```
ubuntu@Deploy-server:~/angularCalc$ cd
ubuntu@Deploy-server:~$ sudo rm -rf /var/www/html/*
ubuntu@Deploy-server:~$ sudo cp -r angularCalc/* /var/www/html/
ubuntu@Deploy-server:~$ sudo systemctl restart nginx
ubuntu@Deploy-server:~$ ls
angularCalc  apache-tomcat-9.0.118  apache-tomcat-9.0.118.tar.gz  calculator-v1.zip  devops.pem
```

Conclusion

This project successfully implements a manual three-server architecture using separate servers for building, storing, and deploying the application.

The Build Server handles source code compilation and packaging. The Nexus Repository Server securely stores the generated artifacts. The Deployment Server downloads and deploys the application.

This architecture improves organization, scalability, artifact management, and deployment reliability by separating responsibilities across multiple servers. All operations such as build, upload, download, and deployment are performed manually.